

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación gráfica, visión computacional y multimedia				
TÍTULO DE LA PRÁCTICA:	Procesamiento de Imágenes				
NÚMERO DE PRÁCTICA:	07	AÑO LECTIVO:	2026-A	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN	17/06/2026	HORA DE PRESENTACIÓN	23:59 pm		
INTEGRANTE (s): Perez Huamani Jeremy Joshua				NOTA:	
DOCENTE(s): Prof. René Nieto					

SOLUCIÓN Y RESULTADOS

I. Ejercicios Propuestos

El presente documento desarrolla las operaciones solicitadas en la practica: redimensionamiento, combinacion de canales RGB, negativo, escala de grises, visualizacion interactiva de canales, dibujo de figuras, umbral binario y programa de dibujo con mouse y teclado.

1. Objetivo del proyecto El objetivo es implementar un programa de procesamiento de imagenes que permita cargar imagenes a color, manipular sus dimensiones y canales, generar nuevas imagenes a partir de operaciones pixel a pixel y crear pequenas aplicaciones interactivas usando OpenCV.

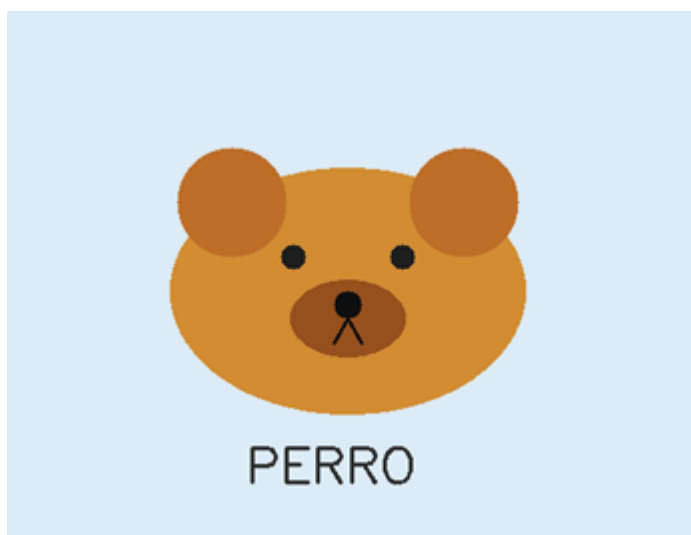
2. Herramientas utilizadas

Herramienta	Uso en el proyecto
Python	Lenguaje principal de implementacion.
OpenCV	Lectura, escritura, conversion, redimensionamiento, threshold, eventos de teclado y mouse
NumPy	Manejo de matrices que representan las imagenes digitales.

3. Imágenes de entrada

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Se utilizaron tres imágenes a color: una persona, un perro y un gato. Cada imagen se interpreta como una matriz de pixeles en formato BGR, que es el formato con el que trabaja OpenCV internamente



**GATO**

4. Desarrollo y resultados

4.1 Redimensionamiento

Primero se abren las tres imagenes y se calcula cual tiene mayor area. Luego todas se redimensionan al mismo ancho y alto de la imagen mas grande. En este caso, el tamaño final usado fue 520 x 400 pixeles

**PERSONA**



PERRO



GATO

4.2 Combinación de canales de color

Se creó una nueva imagen usando solamente el canal rojo de la primera imagen, el canal verde de la segunda imagen y el canal azul de la tercera imagen. Debido a que OpenCV usa BGR, se extrae R con índice 2, G con índice 1 y B con índice 0.

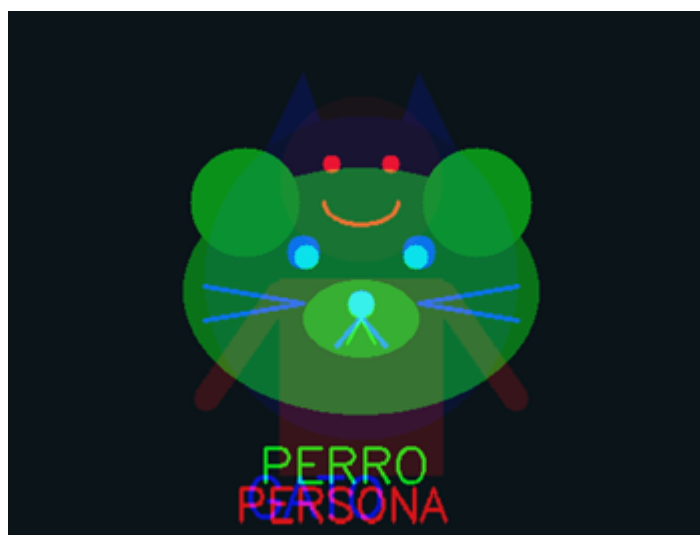
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5



Resultado de combinar R de la primera imagen, G de la segunda y B de la tercera.

4.3 Conversión a negativo y escala de grises

Para obtener el negativo se invierte cada valor de color con la operación $255 - \text{pixel}$. Luego, esa imagen se convierte a escala de grises usando `cv2.cvtColor` con `COLOR_BGR2GRAY`.



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6



4.4 Aplicación interactiva de visualización de canales

La aplicación muestra una imagen y permite activar o desactivar los canales rojo, verde y azul con las teclas R, G y B. Internamente separa los canales con `cv2.split`, reemplaza los canales desactivados por matrices de ceros y reconstruye la imagen con `cv2.merge`.

Python

```
def visualizador_canales(img):
    mostrar_r, mostrar_g, mostrar_b = True, True, True
    while True:
        b, g, r = cv2.split(img)
        cero = np.zeros_like(b)
        salida = cv2.merge([
            b if mostrar_b else cero,
            g if mostrar_g else cero,
            r if mostrar_r else cero,
        ])
        cv2.imshow('Visualizador de canales', salida)
        tecla = cv2.waitKey(30) & 0xFF
        if tecla == 27: break
        elif tecla in [ord('r'), ord('R')]: mostrar_r = not mostrar_r
        elif tecla in [ord('g'), ord('G')]: mostrar_g = not mostrar_g
        elif tecla in [ord('b'), ord('B')]: mostrar_b = not mostrar_b
```

4.5 Dibujo de figuras y texto en imagen

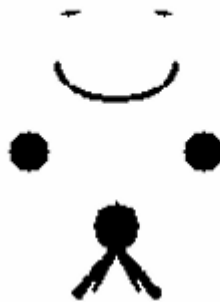
Se seleccionó la imagen de la persona y se dibujó un círculo sobre la zona de la cara. También se agregó texto descriptivo usando `cv2.putText`. Finalmente, el resultado se guardó en la carpeta outputs.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7



4.6 Aplicación de umbral binario.

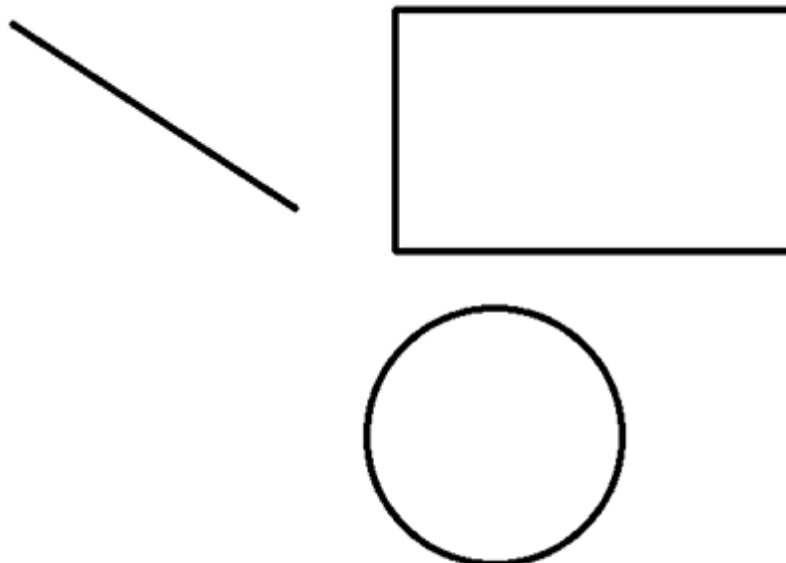
El umbral binario convierte la imagen a blanco y negro segun un valor limite. Primero se convierte la imagen a escala de grises y luego se aplica cv2.threshold. Los pixeles mayores al umbral se vuelven blancos y los menores se vuelven negros.



PERRO

4.7 Programa de dibujo interactivo con mouse y teclado

Se desarrolló una clase llamada Dibujado Interactivo. El usuario puede dibujar líneas, rectángulos y círculos usando eventos de mouse. También se agregó un historial de lienzos para permitir deshacer cambios con la tecla Z y guardar el dibujo con la tecla S.



Ejemplo de dibujo interactivo

Anexo:

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Python

"""

Laboratorio 7 - Procesamiento de Imágenes

Autor: Jeremy Perez

Curso: Computación Gráfica, Visión Computacional y Multimedia

Este programa resuelve las operaciones solicitadas:

1. Abrir 3 imágenes a color.
2. Redimensionarlas al tamaño de la imagen más grande.
3. Crear una imagen nueva combinando canales: R de imagen 1, G de imagen 2 y B de imagen 3.
4. Convertir la imagen combinada a negativo y escala de grises.
5. Visualizador interactivo de canales R, G, B.
6. Dibujar círculo y texto sobre una imagen.
7. Aplicar umbral binario.
8. Crear programa de dibujo interactivo con mouse y teclado, con deshacer y guardar.

Requisitos:

pip install opencv-python numpy

Ejecución:

python main.py

Controles del visualizador de canales:

R = activar/desactivar canal rojo
G = activar/desactivar canal verde
B = activar/desactivar canal azul
ESC = salir

Controles del programa de dibujo:

1 = modo línea
2 = modo rectángulo
3 = modo círculo
Z = deshacer
S = guardar
C = limpiar lienzo
ESC = salir

"""

import os

import cv2

import numpy as np

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

IMG_DIR = os.path.join(BASE_DIR, "imagenes")

OUT_DIR = os.path.join(BASE_DIR, "outputs")

os.makedirs(IMG_DIR, exist_ok=True)

os.makedirs(OUT_DIR, exist_ok=True)

```
def crear_imagenes_demo():
    """Crea 3 imagenes de ejemplo con persona/animales si no existen."""
    rutas = [
        os.path.join(IMG_DIR, "persona.png"),
        os.path.join(IMG_DIR, "perro.png"),
        os.path.join(IMG_DIR, "gato.png"),
    ]
    if all(os.path.exists(r) for r in rutas):
        return rutas

    # Imagen 1: persona
    img1 = np.full((360, 480, 3), (225, 235, 245), dtype=np.uint8)
    cv2.circle(img1, (240, 115), 55, (70, 160, 230), -1) # cara
    cv2.circle(img1, (220, 105), 6, (20, 20, 20), -1)
    cv2.circle(img1, (260, 105), 6, (20, 20, 20), -1)
    cv2.ellipse(img1, (240, 130), (25, 15), 0, 0, 180, (20, 20, 20), 2)
    cv2.rectangle(img1, (185, 180), (295, 310), (80, 80, 200), -1)
    cv2.line(img1, (185, 190), (135, 260), (80, 80, 200), 16)
    cv2.line(img1, (295, 190), (345, 260), (80, 80, 200), 16)
    cv2.putText(img1, "PERSONA", (155, 340), cv2.FONT_HERSHEY_SIMPLEX, 1, (40, 40, 40),
2)
    cv2.imwrite(rutas[0], img1)

    # Imagen 2: perro
    img2 = np.full((400, 520, 3), (245, 235, 220), dtype=np.uint8)
    cv2.ellipse(img2, (260, 210), (130, 90), 0, 0, 360, (50, 140, 210), -1)
    cv2.circle(img2, (175, 145), 40, (40, 110, 190), -1)
    cv2.circle(img2, (345, 145), 40, (40, 110, 190), -1)
    cv2.circle(img2, (220, 185), 9, (30, 30, 30), -1)
    cv2.circle(img2, (300, 185), 9, (30, 30, 30), -1)
    cv2.ellipse(img2, (260, 230), (42, 28), 0, 0, 360, (30, 80, 150), -1)
    cv2.circle(img2, (260, 220), 10, (15, 15, 15), -1)
    cv2.line(img2, (260, 230), (250, 248), (15, 15, 15), 2)
    cv2.line(img2, (260, 230), (270, 248), (15, 15, 15), 2)
    cv2.putText(img2, "PERRO", (185, 350), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (45, 45, 45),
2)
    cv2.imwrite(rutas[1], img2)

    # Imagen 3: gato
    img3 = np.full((320, 430, 3), (230, 245, 230), dtype=np.uint8)
    pts1 = np.array([[145, 110], [180, 40], [210, 125]], np.int32)
    pts2 = np.array([[285, 110], [250, 40], [220, 125]], np.int32)
    cv2.fillPoly(img3, [pts1], (190, 120, 60))
    cv2.fillPoly(img3, [pts2], (190, 120, 60))
    cv2.circle(img3, (215, 160), 95, (205, 145, 80), -1)
    cv2.circle(img3, (180, 145), 10, (20, 20, 20), -1)
    cv2.circle(img3, (250, 145), 10, (20, 20, 20), -1)
    cv2.circle(img3, (215, 175), 8, (20, 20, 20), -1)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

cv2.line(img3, (215, 183), (200, 200), (20, 20, 20), 2)
cv2.line(img3, (215, 183), (230, 200), (20, 20, 20), 2)
for y in [165, 185]:
    cv2.line(img3, (120, y), (180, 175), (20, 20, 20), 2)
    cv2.line(img3, (250, 175), (310, y), (20, 20, 20), 2)
cv2.putText(img3, "GATO", (145, 300), cv2.FONT_HERSHEY_SIMPLEX, 1.1, (40, 40, 40), 2)
cv2.imwrite(rutas[2], img3)
return rutas

def abrir_imagenes(rutas):
    """Lee imagenes desde disco usando OpenCV."""
    imagenes = []
    for ruta in rutas:
        img = cv2.imread(ruta)
        if img is None:
            raise FileNotFoundError(f"No se pudo abrir la imagen: {ruta}")
        imagenes.append(img)
    return imagenes

def redimensionar_a_mayor(imagenes):
    """Redimensiona todas las imagenes al ancho y alto de la imagen con mayor area."""
    mayor = max(imagenes, key=lambda img: img.shape[0] * img.shape[1])
    alto, ancho = mayor.shape[:2]
    redimensionadas = [cv2.resize(img, (ancho, alto), interpolation=cv2.INTER_AREA) for
img in imagenes]
    return redimensionadas, (ancho, alto)

def combinar_canales(imagenes):
    """Crea nueva imagen con R de imagen 1, G de imagen 2 y B de imagen 3."""
    img1, img2, img3 = imagenes
    # OpenCV trabaja en orden BGR, por eso: B=indice 0, G=indice 1, R=indice 2.
    canal_rojo = img1[:, :, 2]
    canal_verde = img2[:, :, 1]
    canal_azul = img3[:, :, 0]
    combinada = cv2.merge([canal_azul, canal_verde, canal_rojo])
    return combinada

def convertir_negativo(img):
    """Invierte los colores: nuevo_pixel = 255 - pixel."""
    return 255 - img

def convertir_grises(img):
    """Convierte una imagen BGR a escala de grises."""
    return cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```
def dibujar_circulo_y_texto(img):
    """Dibuja un círculo sobre la cara de la figura y agrega texto descriptivo."""
    resultado = img.copy()
    alto, ancho = resultado.shape[:2]
    centro = (ancho // 2, alto // 3)
    radio = min(ancho, alto) // 5
    cv2.circle(resultado, centro, radio, (0, 0, 255), 4)
    cv2.putText(resultado, "Figura: Persona", (30, alto - 35),
                  cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 3)
    return resultado

def aplicar_umbral_binario(img, valor_umbral=127):
    """Convierte la imagen a blanco y negro con threshold binario."""
    gris = convertir_grises(img)
    _, binaria = cv2.threshold(gris, valor_umbral, 255, cv2.THRESH_BINARY)
    return binaria

def visualizador_canales(img):
    """Aplicacion interactiva para activar/desactivar canales R, G y B."""
    mostrar_r = True
    mostrar_g = True
    mostrar_b = True

    while True:
        b, g, r = cv2.split(img)
        cero = np.zeros_like(b)
        salida = cv2.merge([
            b if mostrar_b else cero,
            g if mostrar_g else cero,
            r if mostrar_r else cero,
        ])
        texto = f"R:{mostrar_r} G:{mostrar_g} B:{mostrar_b} | R/G/B alternan | ESC sale"
        cv2.putText(salida, texto, (15, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 255), 2)
        cv2.imshow("Visualizador de canales", salida)
        tecla = cv2.waitKey(30) & 0xFF
        if tecla == 27:
            break
        elif tecla in [ord('r'), ord('R')]:
            mostrar_r = not mostrar_r
        elif tecla in [ord('g'), ord('G')]:
            mostrar_g = not mostrar_g
        elif tecla in [ord('b'), ord('B')]:
            mostrar_b = not mostrar_b
    cv2.destroyAllWindows()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

class DibujadorInteractivo:
    """Programa para dibujar lineas, rectangulos y circulos con mouse y teclado."""
    def __init__(self, ancho=800, alto=550):
        self.lienzo = np.full((alto, ancho, 3), 255, dtype=np.uint8)
        self.historial = []
        self.modos = "linea"
        self.dibujando = False
        self.inicio = None
        self.temporal = self.lienzo.copy()

    def mouse_callback(self, event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN:
            self.historial.append(self.lienzo.copy())
            self.dibujando = True
            self.inicio = (x, y)
        elif event == cv2.EVENT_MOUSEMOVE and self.dibujando:
            self.temporal = self.lienzo.copy()
            self._dibujar_figura(self.temporal, self.inicio, (x, y))
        elif event == cv2.EVENT_LBUTTONUP:
            self.dibujando = False
            self._dibujar_figura(self.lienzo, self.inicio, (x, y))
            self.temporal = self.lienzo.copy()

    def _dibujar_figura(self, img, p1, p2):
        color = (0, 0, 0)
        grosor = 3
        if self.modos == "linea":
            cv2.line(img, p1, p2, color, grosor)
        elif self.modos == "rectangulo":
            cv2.rectangle(img, p1, p2, color, grosor)
        elif self.modos == "circulo":
            radio = int(np.sqrt((p2[0] - p1[0]) ** 2 + (p2[1] - p1[1]) ** 2))
            cv2.circle(img, p1, radio, color, grosor)

    def ejecutar(self):
        ventana = "Dibujo interactivo"
        cv2.namedWindow(ventana)
        cv2.setMouseCallback(ventana, self.mouse_callback)

        while True:
            vista = self.temporal.copy()
            ayuda = "1 Linea | 2 Rectangulo | 3 Circulo | Z Deshacer | S Guardar | C
Limpia | ESC Salir"
            cv2.putText(vista, f"Modo: {self.modos}", (15, 30), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (0, 0, 255), 2)
            cv2.putText(vista, ayuda, (15, 65), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (60, 60,
60), 2)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

cv2.imshow(ventana, vista)
tecla = cv2.waitKey(20) & 0xFF

if tecla == 27:
    break
elif tecla == ord('1'):
    self.moda = "linea"
elif tecla == ord('2'):
    self.moda = "rectangulo"
elif tecla == ord('3'):
    self.moda = "circulo"
elif tecla in [ord('z'), ord('Z')]:
    if self.historial:
        self.lienzo = self.historial.pop()
        self.temporal = self.lienzo.copy()
elif tecla in [ord('s'), ord('S')]:
    ruta = os.path.join(OUT_DIR, "dibujo_final.png")
    cv2.imwrite(ruta, self.lienzo)
    print(f"Dibujo guardado en: {ruta}")
elif tecla in [ord('c'), ord('C')]:
    self.historial.append(self.lienzo.copy())
    self.lienzo[:] = 255
    self.temporal = self.lienzo.copy()
cv2.destroyAllWindows()

def generar_resultados(no_interactivo=True):
    """Ejecuta las operaciones principales y guarda las imagenes resultado."""
    rutas = crear_imagenes_demo()
    imagenes = abrir_imagenes(rutas)

    redimensionadas, tam = redimensionar_a_mayor(imagenes)
    for i, img in enumerate(redimensionadas, start=1):
        cv2.imwrite(os.path.join(OUT_DIR, f"imagen_{i}_redimensionada.png"), img)

    combinada = combinar_canales(redimensionadas)
    cv2.imwrite(os.path.join(OUT_DIR, "imagen_combinada_rgb.png"), combinada)

    negativa = convertir_negativo(combinada)
    cv2.imwrite(os.path.join(OUT_DIR, "imagen_negativa.png"), negativa)

    gris = convertir_grises(negativa)
    cv2.imwrite(os.path.join(OUT_DIR, "imagen_grises.png"), gris)

    anotada = dibujar_circulo_y_texto(redimensionadas[0])
    cv2.imwrite(os.path.join(OUT_DIR, "imagen_circulo_texto.png"), anotada)

    binaria = aplicar_umbral_binario(combinada, 127)
    cv2.imwrite(os.path.join(OUT_DIR, "imagen_umbral_binario.png"), binaria)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```
# Ejemplo no interactivo del dibujo final para evidenciar el resultado.
lienzo = np.full((550, 800, 3), 255, dtype=np.uint8)
cv2.line(lienzo, (80, 100), (280, 230), (0, 0, 0), 3)
cv2.rectangle(lienzo, (350, 90), (630, 260), (0, 0, 0), 3)
cv2.circle(lienzo, (420, 390), 90, (0, 0, 0), 3)
cv2.putText(lienzo, "Ejemplo de dibujo interactivo", (80, 510),
cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
cv2.imwrite(os.path.join(OUT_DIR, "dibujo_final_ejemplo.png"), lienzo)

print("Resultados generados correctamente en la carpeta outputs/")
print(f"Todas las imagenes fueron redimensionadas a: {tam[0]}x{tam[1]}")
return combinada

def main():
    combinada = generar_resultados()

    print("\nProyecto resuelto. Para probar partes interactivas, descomenta las lineas
    finales en main().")
    print("- visualizador canales(combinada)")
    print("- DibujadorInteractivo().ejecutar()")

    # Descomentar para ejecutar la aplicacion de canales:
    # visualizador_canales(combinada)

    # Descomentar para ejecutar el programa de dibujo interactivo:
    # DibujadorInteractivo().ejecutar()

if __name__ == "__main__":
    main()
```

II. Cuestionario

1. ¿Qué otros modelos de colores existen para las imágenes?

Además del modelo RGB, existen modelos como HSV, HSL, CMYK, YCrCb, LAB y escala de grises. HSV y HSL son útiles para trabajar con tono, saturación e iluminación. CMYK se usa principalmente en impresión. YCrCb y LAB ayudan a separar luminancia de crominancia, por lo que son útiles en visión computacional.

2. ¿Qué otros cambios se pueden realizar a las imágenes con OpenCV?

Con OpenCV se pueden realizar rotaciones, recortes, traslaciones, cambios de brillo y contraste, desenfoque, detección de bordes, detección de contornos, filtros, ecualización de

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

histograma, segmentacion, operaciones morfologicas, deteccion de objetos y conversiones entre espacios de color.

3. ¿Cuál sería el uso del binary threshold en el procesamiento de imágenes?

El threshold binario se usa para separar objetos del fondo cuando existe una diferencia clara de intensidad. Es muy útil para segmentar formas, aislar texto, separar regiones claras y oscuras, preparar imágenes para detección de contornos y simplificar una imagen antes de aplicar otros algoritmos.

III. CONCLUSIONES

- Se logro implementar un flujo completo de procesamiento de imagenes con OpenCV. El trabajo mostro que una imagen digital puede tratarse como una matriz de pixeles y que, al manipular sus canales y valores, se pueden generar transformaciones visuales como combinaciones RGB, negativos, escala de grises y segmentacion binaria. Tambien se comprobo que OpenCV permite crear aplicaciones interactivas mediante eventos de teclado y mouse

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[\[https://www.youtube.com/watch?v=GbmRt0wydQU\]](https://www.youtube.com/watch?v=GbmRt0wydQU)
[\[https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOK0YHJ7il1eY2vFT9pM9e1%2fh1M%3d\]](https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOK0YHJ7il1eY2vFT9pM9e1%2fh1M%3d)